

Les accesseurs en c#, pratiques 'usuelles'

```
class Person
{
    private string name;
    public string Name
    {
        get
        {
            return name;
        }
        set
        {
            name = value;
        }
    }
}
```

```
public class Date
{
    private int month = 7; // Valeur par défaut

    public int Month
    {
        get
        {
            return month;
        }
        set
        {
            if ((value > 0) && (value < 13))
            {
                month = value;
            }
        }
    }
}
```

using System;

```
class Person
{
    private string name;
    public string Name
    {
        get
        {
            return name;
        }
        set
        {

```

```

        name = value;
    }
}

public class Program
{
    public static void Main()
    {
        Person UnePersonne = new Person();
        UnePersonne.Name = "John";
        Console.WriteLine("Le nom de la personne est "+UnePersonne.Name);
    }
}

```

Travail à faire

Dotez la classe Person d'une propriété motDePasse munie de ses accesseurs.
(Un mot de passe devra obligatoirement être d'une longueur >8 caractères)
Vérifiez le bon fonctionnement dans le programme principal.

1. **Qu'est-ce qu'un constructeur ?** : Un constructeur est une méthode spéciale dans une classe C# qui est appelée lorsqu'un nouvel objet de cette classe est créé. Le constructeur est responsable de l'initialisation des nouvelles instances de la classe.

2. **Types de constructeurs** :

- **Constructeur par défaut** : Un constructeur par défaut est un constructeur qui ne prend pas de paramètres. Si aucun constructeur n'est explicitement défini dans une classe, un constructeur par défaut est automatiquement créé par le compilateur.
- **Constructeur surchargé** : Un constructeur surchargé est un constructeur qui prend un ou plusieurs paramètres. Vous pouvez définir plusieurs constructeurs dans une classe, chacun prenant un ensemble différent de paramètres.

3. **Syntaxe des constructeurs** : Voici la syntaxe de base pour déclarer un constructeur dans une classe C# :

```

public class MaClasse
{
    // Constructeur par défaut
    public MaClasse()
    {

```

```

    // Initialisation
}

// Constructeur surchargé
public MaClasse(int parametre1, string parametre2)
{
    // Initialisation avec les paramètres
}
}

```

Les constructeurs et leur usage en c#

Les constructeurs sont des éléments fondamentaux de la programmation orientée objet. Ils sont utilisés pour créer de nouvelles instances de classes et initialiser leurs états initiaux.

1. Utilisation des constructeurs :

- Lorsque vous créez une nouvelle instance d'une classe en utilisant l'opérateur `new`, le constructeur approprié est automatiquement appelé.
- Vous pouvez appeler un constructeur surchargé en fonction des paramètres que vous fournissez lors de la création de l'objet.

2. Initialisation d'instances :

- Les constructeurs sont souvent utilisés pour initialiser les champs de données d'une classe avec des valeurs par défaut.
- Vous pouvez également effectuer d'autres opérations d'initialisation dans un constructeur, telles que l'ouverture de fichiers, la connexion à une base de données, etc.

Les constructeurs sont des éléments fondamentaux de la programmation orientée objet en C#. Ils sont utilisés pour créer de nouvelles instances de classes et initialiser leurs états initiaux.

4. **Qu'est-ce qu'un constructeur ?** : Un constructeur est une méthode spéciale dans une classe C# qui est appelée lorsqu'un nouvel objet de cette classe est créé. Le constructeur est responsable de l'initialisation des nouvelles instances de la classe.

5. Types de constructeurs :

- **Constructeur par défaut** : Un constructeur par défaut est un constructeur qui ne prend pas de paramètres. Si aucun constructeur n'est explicitement défini dans une classe, un constructeur par défaut est automatiquement créé par le compilateur.
- **Constructeur surchargé** : Un constructeur surchargé est un constructeur qui prend un ou plusieurs paramètres. Vous pouvez définir plusieurs constructeurs dans une classe, chacun prenant un ensemble différent de paramètres.

6. Syntaxe des constructeurs : Voici la syntaxe de base pour déclarer un constructeur dans une classe C# :

```
public class MaClasse
{
    // Constructeur par défaut
    public MaClasse()
    {
        // Initialisation
    }

    // Constructeur surchargé
    public MaClasse(int parametre1, string parametre2)
    {
        // Initialisation avec les paramètres
    }
}
```

3. Utilisation des constructeurs :

- Lorsque vous créez une nouvelle instance d'une classe en utilisant l'opérateur `new`, le constructeur approprié est automatiquement appelé.
- Vous pouvez appeler un constructeur surchargé en fonction des paramètres que vous fournissez lors de la création de l'objet.

4. Initialisation d'instances :

- Les constructeurs sont souvent utilisés pour initialiser les champs de données d'une classe avec des valeurs par défaut.
- Vous pouvez également effectuer d'autres opérations d'initialisation dans un constructeur, telles que l'ouverture de fichiers, la connexion à une base de données, etc.

Travail à faire

Dotez la classe Person d'un constructeur par défaut.

Ce constructeur par défaut aura pour effet d'initialiser le nom de la personne à la chaîne vide et le mot de passe à une valeur attribuée automatiquement (Vous utiliserez pour cela la méthode GeneratePassword vue dans le contrôle de connaissances).

Vérifiez dans le programme principal le bon fonctionnement de ce constructeur par défaut.

Dotez la classe Person d'un constructeur Person (LeNom : chaîne, LePass : chaîne) et vérifiez son bon fonctionnement.

Faites en sorte que quoi qu'il arrive, le mot de passe contienne bien au moins 10 caractères.

Corrigé

```
using System;
public class Program
{
    class Person
    {
        public Person() // Constructeur par défaut
        {
            name="";
            Random random = new Random();
            mdp=GeneratePassword(random.Next(10, 30));
        }

        public Person(string leNom, String lePass) // Constructeur surchargé
        {
            Name=leNom;
            MPD=lePass;
        }

        private string GeneratePassword(int length)
        {
            const string chars =
"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz";
            char[] password = new char[length];
            Random random = new Random();
            for (int i = 0; i < length; i++)
            {
                password[i] = chars[random.Next(chars.Length)];
            }
            return new string(password);
        }

        private string name;
        public string Name
```

```

{
    get
    {
        return name;
    }
    set
    {
        name = value;
    }
}

        private string mdp;
public string MPD
{
    get
    {
        return mdp;
    }
    set
    {
        if (value.Length>=10)
            name = value;
    }
}
}

    public static void Main()
    {
        Person UnePersonne =new Person();
        UnePersonne.Name="John";
        Console.WriteLine("Le nom de la personne est
"+UnePersonne.Name);
        Console.WriteLine("Son mot de passe est "+UnePersonne.MPD);
        Console.WriteLine("----- ");
        Person UneAutrePersonne =new Person();
        UneAutrePersonne.Name="Bill";
        Console.WriteLine("Le nom de la seconde personne est
"+UneAutrePersonne.Name);
        Console.WriteLine("Son mot de passe est
"+UneAutrePersonne.MPD);
        UneAutrePersonne.MPD="Bidon";
        Console.WriteLine("Le mot de passe de
"+UneAutrePersonne.Name+" est maintenant "+UneAutrePersonne.MPD);

    }
}

```